

Module 4: Node.js, Express & TypeORM › Section 14: Error Handling, Testing & Deployment › 14-01
Error Handling

Error Handling: Centralised Middleware, Custom Error Classes & Logging

SECTION INFO

Section: 14 | Subtopic: 01 | Estimated Duration: 2–3 hours

Theory

Practical

Topics covered: Centralised error-handling middleware in Express, custom error classes with HTTP status codes, structured logging with Winston, request logging with Morgan, uncaught exception and unhandled rejection handling

Learning Objectives — This Subtopic

- Implement a four-argument Express error-handling middleware that intercepts all errors thrown in route handlers and other middleware
- Design a custom error class hierarchy that carries HTTP status codes, making error responses consistent and predictable
- Distinguish between operational errors (expected, recoverable) and programmer errors (bugs), and handle each appropriately
- Configure Winston to produce structured JSON logs in production and human-readable logs in development, with appropriate log levels
- Use Morgan to generate per-request access logs and integrate them with Winston as the output transport
- Register global handlers for `uncaughtException` and `unhandledRejection` to ensure no error escapes silently

1. The Problem with Scattered Error Handling

Without a deliberate error strategy, each route handler tends to grow its own ad-hoc error response logic: one returns a bare string, another a JSON object with a `message` key, a third throws an unhandled exception that crashes the process. The result is an API where error responses are unpredictable in shape, status codes are inconsistent, and failures are invisible in production logs.

Express provides a formal mechanism for solving this: the **four-argument error-handling middleware**. Every other approach — try/catch in every handler, per-route error responses — is a workaround for not using it properly.

Analogy: Think of a hospital's triage system. Individual wards do not independently decide how to handle emergencies that exceed their capacity — they escalate to a central emergency department with trained staff, standard procedures, and proper records. Centralised error handling is the same pattern: individual route handlers detect problems and escalate; one place decides how to respond and what to log.

2. Project Structure

```
src/ |— server.ts # Entry point – starts HTTP server, global handlers |— app.ts # Express app
factory – middleware registration |— errors/ |   |— AppError.ts # Custom error class hierarchy
|— middleware/ |   |— errorHandler.ts # Centralised error-handling middleware |— logger/ |   |—
logger.ts # Winston instance |   |— routes/ |   |— users.ts # Example route used in practicals
```

```
npm install express winston morgan
```

```
npm install --save-dev typescript @types/node @types/express @types/morgan ts-node-dev
```

3. Custom Error Classes

3.1 Why Native `Error` Is Not Enough

JavaScript's built-in `Error` carries a `message` and a `stack` trace. It carries nothing about *what HTTP response to send*. A route handler that throws `new Error('User not found')` provides no signal about whether the appropriate status code is 404, 400, or 500. The error handler is forced to guess, or worse, always returns 500.

A custom error class adds the fields the error handler needs to construct an accurate, consistent response without guessing.

3.2 Stage 1 — A Single `AppError` Class

STAGE 1

`src/errors/AppError.ts`

```
export class AppError extends Error {
  public readonly statusCode: number;
  public readonly isOperational: boolean;

  constructor(message: string, statusCode: number, isOperational = true) {
    super(message);

    // Restore the prototype chain – required when extending built-ins in TypeScript
    Object.setPrototypeOf(this, new.target.prototype);

    this.statusCode = statusCode;
    this.isOperational = isOperational;
    Error.captureStackTrace(this, this.constructor);
  }
}
```

Two fields drive everything downstream:

- `statusCode` — the HTTP status the error handler sends to the client.
- `isOperational` — a flag that distinguishes expected failures (user not found, invalid input) from unexpected programmer errors (undefined is not a function). Operational errors are safe to describe to the client; non-operational errors indicate a bug and should return a generic 500 without leaking details.

3.3 Stage 2 — A Domain-Specific Error Hierarchy

Rather than passing magic numbers throughout the codebase, derive purpose-built subclasses. The constructor sets the status code so call sites read like prose.

STAGE 2

src/errors/AppError.ts

```
export class AppError extends Error {
  public readonly statusCode: number;
  public readonly isOperational: boolean;

  constructor(message: string, statusCode: number, isOperational = true) {
    super(message);
    Object.setPrototypeOf(this, new.target.prototype);
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    Error.captureStackTrace(this, this.constructor);
  }
}

export class NotFoundError extends AppError {
  constructor(resource = 'Resource') {
    super(`${resource} not found.`, 404);
  }
}

export class ValidationError extends AppError {
  constructor(message: string) {
    super(message, 400);
  }
}

export class UnauthorisedError extends AppError {
  constructor(message = 'Authentication required.') {
    super(message, 401);
  }
}

export class ForbiddenError extends AppError {
  constructor(message = 'You do not have permission to perform this action.') {
    super(message, 403);
  }
}

export class ConflictError extends AppError {
```

```
constructor(message: string) {  
  super(message, 409);  
}  
}
```

A route handler now reads cleanly:

src/routes/users.ts (excerpt)

```
import { Router, Request, Response, NextFunction } from 'express';  
import { NotFoundError, ValidationError } from '../errors/AppError';  
import { AppDataSource } from '../data-source';  
import { User } from '../entities/User';  
  
const router = Router();  
const userRepo = AppDataSource.getRepository(User);  
  
router.get('/:id', async (req: Request, res: Response, next: NextFunction) => {  
  const id = parseInt(req.params.id, 10);  
  
  if (isNaN(id)) {  
    return next(new ValidationError('User ID must be a valid integer.'));  
  }  
  
  const user = await userRepo.findOneBy({ id });  
  
  if (!user) {  
    return next(new NotFoundError('User'));  
  }  
  
  res.json(user);  
});  
  
export default router;
```

Output:

```
GET /users/abc → 400 Bad Request { "error": "User ID must be a valid integer." }  
GET /users/999 → 404 Not Found { "error": "User not found." }  
GET /users/1 → 200 OK { "id": 1, "name": "Alice", ... }
```

Key Insight: Notice that every async route calls `next(error)` rather than throwing directly. Express's error handler only intercepts errors passed to `next()`. An unhandled promise rejection inside an async route handler in Express 4 will *not* automatically reach the error middleware — you must either call `next(err)` in a catch block or wrap your routes. Express 5 (currently in beta as of early 2026) changes this behaviour, but the explicit `next(err)` pattern is compatible with both versions and makes intent clear.

3.4 Async Route Wrapper Utility

Repeating `try { ... } catch (err) { next(err) }` in every async handler is mechanical and easy to forget. Extract it into a wrapper:

`src/errors/asyncHandler.ts`

```
import { Request, Response, NextFunction, RequestHandler } from 'express';

type AsyncRequestHandler = (
  req: Request,
  res: Response,
  next: NextFunction
) => Promise<void | Response>;

/**
 * Wraps an async route handler so that any rejected promise
 * is automatically forwarded to Express's error middleware via next().
 */
export const asyncHandler =
  (fn: AsyncRequestHandler): RequestHandler =>
    (req, res, next) =>
      Promise.resolve(fn(req, res, next)).catch(next);
```

`src/routes/users.ts (using asyncHandler)`

```
import { Router, Request, Response } from 'express';
import { asyncHandler } from '../errors/asyncHandler';
import { NotFoundError, ValidationError } from '../errors/AppError';
import { AppDataSource } from '../data-source';
import { User } from '../entities/User';

const router = Router();
const userRepo = AppDataSource.getRepository(User);
```

```
router.get(
 ('/:id',
  asyncHandler(async (req: Request, res: Response) => {
    const id = parseInt(req.params.id, 10);

    if (isNaN(id)) throw new ValidationError('User ID must be a valid integer.');
```



```
    const user = await userRepo.findOneBy({ id });

    if (!user) throw new NotFoundError('User');

    res.json(user);
  })
);

export default router;
```

Output:

```
GET /users/abc → 400 { "error": "User ID must be a valid integer." }
GET /users/999 → 404 { "error": "User not found." }
GET /users/1 → 200 { "id": 1, "name": "Alice" }
```

With `asyncHandler` in place, route handlers can throw freely — any thrown error, whether an `AppError` subclass or an unexpected runtime exception, propagates to the centralised error handler automatically.

4. Centralised Error-Handling Middleware

4.1 The Four-Argument Signature

Express identifies error-handling middleware by its arity: the function must declare exactly four parameters — `(err, req, res, next)`. Express will not route errors to a three-argument middleware, even if its name suggests otherwise. This middleware must be registered *after all routes*.

4.2 Stage 1 — Basic Error Handler

STAGE 1

src/middleware/errorHandler.ts

```
import { Request, Response, NextFunction } from 'express';
import { AppError } from '../errors/AppError';

export function errorHandler(
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction
): void {
  if (err instanceof AppError) {
    res.status(err.statusCode).json({ error: err.message });
    return;
  }

  // Unknown/programmer error – do not leak internals
  console.error('Unexpected error:', err);
  res.status(500).json({ error: 'An unexpected error occurred.' });
}
```

src/app.ts

```
import express, { Application } from 'express';
import userRouter from './routes/users';
import { errorHandler } from './middleware/errorHandler';

const app: Application = express();

app.use(express.json());
app.use('/users', userRouter);

// Error handler MUST be last
app.use(errorHandler);

export default app;
```


4.3 Stage 2 — Differentiating Operational vs. Programmer Errors

Operational errors are expected conditions: a user does not exist, a required field is missing, a third-party service is temporarily unavailable. The client receives a meaningful error message.

Programmer errors are bugs: a variable accessed before it is defined, a TypeORM method called without an initialised connection, a malformed SQL query. These indicate something is wrong with the application code. The client receives a generic 500; the full stack trace goes to the log.

STAGE 2

src/middleware/errorHandler.ts

```
import { Request, Response, NextFunction } from 'express';
import { AppError } from '../errors/AppError';
import { logger } from '../logger/logger';

interface ErrorResponse {
  error: string;
  stack?: string; // Included in development only
}

export function errorHandler(
  err: Error,
  req: Request,
  res: Response,
  _next: NextFunction
): void {
  const isDev = process.env.NODE_ENV !== 'production';

  if (err instanceof AppError && err.isOperational) {
    // Operational: safe to surface to the client
    logger.warn({
      message: err.message,
      statusCode: err.statusCode,
      path: req.path,
      method: req.method,
    });

    res.status(err.statusCode).json({ error: err.message });
    return;
  }

  // Programmer error or unexpected failure
  logger.error({
    message: err.message,
```

```
    stack: err.stack,
    path: req.path,
    method: req.method,
  });

  const body: ErrorResponse = { error: 'An unexpected error occurred.' };
  if (isDev) body.stack = err.stack;

  res.status(500).json(body);
}
```

Output:

```
// Operational – NotFoundError (isOperational: true)
GET /users/999 → 404
{ "error": "User not found." }

// Programmer error – TypeError thrown inside route
GET /users/1 → 500 (in production)
{ "error": "An unexpected error occurred." }

GET /users/1 → 500 (in development, NODE_ENV=development)
{ "error": "Cannot read properties of undefined (reading 'id')",
  "stack": "TypeError: Cannot read properties of undefined...\n    at ..." }
```

5. Structured Logging with Winston

5.1 Why `console.log` Is Insufficient in Production

`console.log` writes unstructured strings to stdout. In production, logs are ingested by systems (Datadog, CloudWatch, Elastic) that parse structured data — typically JSON — to enable filtering, alerting, and dashboards. A log line of `"Something went wrong: user not found"` is unsearchable at scale. A JSON object with `{ "level": "warn", "statusCode": 404, "userId": 42, "timestamp": "..." }` is.

Winston is the most widely used Node.js logging library. It separates *transports* (where logs go: console, file, HTTP endpoint) from *formatters* (how they look) and *levels* (severity filtering).

5.2 Log Levels

Winston uses the npm log levels by default, from most to least severe:

Level	Value	Typical Use
<code>error</code>	0	Programmer errors, unhandled exceptions, fatal failures
<code>warn</code>	1	Operational errors surfaced to clients (404, 400, 401)
<code>info</code>	2	Normal lifecycle events: server started, DB connected
<code>http</code>	3	Per-request access logs (used with Morgan)
<code>verbose</code>	4	Detailed flow information for debugging
<code>debug</code>	5	Development-only diagnostic output
<code>silly</code>	6	Trace-level — rarely used in practice

Setting `level: 'warn'` on a transport causes it to log `warn` and `error` only. Setting `level: 'debug'` logs everything.

5.3 Stage 1 — Basic Winston Logger

STAGE 1

`src/logger/logger.ts`

```
import winston from 'winston';

const { combine, timestamp, json, colorize, printf } = winston.format;

// Human-readable format for local development
const devFormat = combine(
```

```
colorize(),
timestamp({ format: 'HH:mm:ss' })),
printf(({ level, message, timestamp: ts, ...meta }) => {
  const extra = Object.keys(meta).length ? ` ${JSON.stringify(meta)} ` : '';
  return `${ts} [${level}] ${message}${extra}`;
})
);

// Machine-readable format for production log ingestion
const prodFormat = combine(
  timestamp(),
  json()
);

export const logger = winston.createLogger({
  level: process.env.LOG_LEVEL ?? (process.env.NODE_ENV === 'production' ? 'warn' : 'debug'),
  format: process.env.NODE_ENV === 'production' ? prodFormat : devFormat,
  transports: [
    new winston.transports.Console(),
  ],
});
```

Now replace the `console.error` calls in the error handler and throughout the application with `logger.*` calls. The logger instance is a module singleton — import it wherever needed.

```
NODE_ENV=development npx ts-node-dev src/server.ts
```

Output:

```
10:42:01 [info]    Server listening on port 3000
10:42:05 [warn]    User not found. {"statusCode":404,"path":"/users/999","method":"GET"}
10:42:09 [error]    Cannot read properties of undefined {"stack":"TypeError: ...","path":"/users/1"}
```

```
NODE_ENV=production npx ts-node-dev src/server.ts
```

Output:

```
{"level":"warn","message":"User not found. ","statusCode":404,"path":"/users/999","method":"GET","timestamp":"2026-03-01T10:42:05.123Z"}
{"level":"error","message":"Cannot read properties of undefined","stack":"TypeError: ...","path":"/users/1","timestamp":"2026-03-01T10:42:09.456Z"}
```

5.4 Stage 2 — Adding a Persistent File Transport

Console output is lost when a container restarts. Writing to log files provides a short-term persistent record; log rotation prevents unbounded disk growth.

STAGE 2

src/logger/logger.ts

```
import winston from 'winston';
import path from 'path';

const { combine, timestamp, json, colorize, printf } = winston.format;

const devFormat = combine(
  colorize(),
  timestamp({ format: 'HH:mm:ss' }),
  printf(({ level, message, timestamp: ts, ...meta }) => {
    const extra = Object.keys(meta).length ? ` ${JSON.stringify(meta)} ` : '';
    return `${ts} [${level}] ${message}${extra}`;
  })
);

const prodFormat = combine(timestamp(), json());

const isProduction = process.env.NODE_ENV === 'production';

const transports: winston.transport[] = [
  new winston.transports.Console({
    silent: isProduction, // Suppress console in production; rely on files
  }),
];

if (isProduction) {
  transports.push(
    new winston.transports.File({
      filename: path.join('logs', 'error.log'),
      level: 'error', // Only errors go here
      maxsize: 5_242_880, // 5 MB
      maxFiles: 5,
      tailable: true,
    }),
    new winston.transports.File({
      filename: path.join('logs', 'combined.log'),
      level: 'warn', // Warn + error
      maxsize: 10_485_760, // 10 MB
      maxFiles: 10,
    })
  );
}
```

```
    tailable: true,
  })
);
}

export const logger = winston.createLogger({
  level: process.env.LOG_LEVEL ?? (isProduction ? 'warn' : 'debug'),
  format: isProduction ? prodFormat : devFormat,
  transports,
});
```

Output:

```
logs/
├─ combined.log    ← warn + error entries in JSON
└─ error.log       ← error entries only in JSON
```

Warning: In containerised deployments (Docker, Kubernetes), writing to local files inside the container is usually wrong — the filesystem is ephemeral and not accessible from outside the pod. In those environments, keep only the Console transport and route stdout to a log aggregator (Fluentd, Loki, CloudWatch Logs). File transports are appropriate for bare-metal or VM deployments where the filesystem is persistent.

6. Request Logging with Morgan

6.1 What Morgan Does

Morgan is HTTP request logging middleware for Express. It fires after each request completes and records method, URL, status code, response time, and response size. Winston handles application events; Morgan handles the HTTP access log. The two are complementary, not alternatives.

By default Morgan writes to stdout (`process.stdout`). To integrate it with Winston so both streams land in the same log pipeline, stream Morgan's output into Winston's `http` level.

6.2 Morgan Integrated with Winston

src/app.ts

```
import express, { Application } from 'express';
import morgan from 'morgan';
import { logger } from '../logger/logger';
import userRouter from '../routes/users';
import { errorHandler } from '../middleware/errorHandler';

const app: Application = express();

// Stream Morgan output into Winston at the 'http' level
const morganStream = {
  write: (message: string) => logger.http(message.trim()),
};

// 'combined' is Apache-style; 'dev' is colourised short format for development
app.use(
  morgan(
    process.env.NODE_ENV === 'production' ? 'combined' : 'dev',
    { stream: morganStream }
  )
);

app.use(express.json());
app.use('/users', userRouter);
app.use(errorHandler);

export default app;
```

Output:

```
10:42:05 [http] GET /users/1 200 4.321 ms - 42
10:42:06 [http] GET /users/999 404 1.102 ms - 26
10:42:07 [http] POST /users 400 0.891 ms - 44
```

Key Insight: Because `http` has a numeric value of 3 in Winston's level hierarchy, setting the logger's level to `'warn'` (value 1) in production will suppress all Morgan access logs. If you want access logs in production, set `LOG_LEVEL=http` in your environment, or create a dedicated transport at the `http` level with a separate log file.

7. Global Handlers — Escaping the Express Boundary

Express's error middleware only catches errors that pass through Express's middleware chain. Two categories of failure escape it entirely:

- **`uncaughtException`** — a synchronous error thrown outside any request handler (e.g., during startup, in a timer callback, or in code that was not wrapped by Express).
- **`unhandledRejection`** — a promise that rejected without a `.catch()` handler or a try/catch in an `async` function (common in event-driven code outside route handlers).

Without handlers for these events, Node.js prints the error to stderr and exits with code 1 in newer versions — or, in older versions, silently continues with the process in an undefined state. Either outcome is unacceptable in production.

`src/server.ts`

```
import app from './app';
import { logger } from './logger/logger';
import { AppDataSource } from './data-source';

const PORT = parseInt(process.env.PORT ?? '3000', 10);

// — Global error handlers – registered before anything else —————

process.on('uncaughtException', (err: Error) => {
  logger.error({
    message: 'UNCAUGHT EXCEPTION – shutting down',
    error: err.message,
    stack: err.stack,
  });
  // Give the logger time to flush, then exit with a non-zero code
```

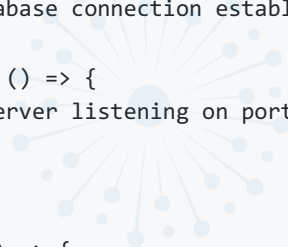


```
// so a process manager (PM2, Kubernetes) restarts the application
process.exit(1);
});

process.on('unhandledRejection', (reason: unknown) => {
  logger.error({
    message: 'UNHANDLED REJECTION – shutting down',
    reason: String(reason),
  });
  process.exit(1);
});

// — Startup —————

async function start(): Promise<void> {
  await AppDataSource.initialize();
  logger.info('Database connection established.');
```



```
  app.listen(PORT, () => {
    logger.info(`Server listening on port ${PORT}`);
  });
}

start().catch((err) => {
  logger.error({ message: 'Failed to start server', error: err.message });
  process.exit(1);
});
```

Output:

```
// If a timer fires and throws after startup:
{"level":"error","message":"UNCAUGHT EXCEPTION – shutting down",
 "error":"Cannot read properties of null","stack":"TypeError: ...","timestamp":"..."}
// Process exits with code 1 → PM2/Kubernetes restarts it
```

Warning: Do not attempt to resume normal operation after an `uncaughtException`. The Node.js documentation explicitly states that the process may be in an inconsistent state after such an event — continuing to serve requests risks data corruption. Exit immediately, log the error, and let the process manager restart a clean instance.

8. Putting It All Together

The final application has three clearly separated concerns:

Concern	File	Responsibility
Error types	<code>src/errors/AppError.ts</code>	Domain-specific error classes with status codes and the <code>isOperational</code> flag
Async safety	<code>src/errors/asyncHandler.ts</code>	Wrapper that forwards any rejected promise to Express's <code>next()</code>
Error response	<code>src/middleware/errorHandler.ts</code>	Single four-argument middleware that formats and sends all error responses
Logging	<code>src/logger/logger.ts</code>	Winston instance with environment-appropriate format and transports
Access logging	<code>src/app.ts</code>	Morgan streaming to Winston's <code>http</code> level
Global safety net	<code>src/server.ts</code>	<code>uncaughtException</code> and <code>unhandledRejection</code> handlers that log and exit

A request that hits an error flows as follows:

Output:

```
Route handler throws NotFoundError('User')
  → asyncHandler catches the rejection
  → calls next(err)
  → Express locates the four-argument errorHandler
  → errorHandler checks err instanceof AppError → true, isOperational → true
```

```

→ logger.warn({ message, statusCode, path })
→ res.status(404).json({ error: 'User not found.' })
→ Morgan logs: GET /users/999 404 1.2ms

```

Summary

Concept	Key Point
Four-argument error middleware	The function signature <code>(err, req, res, next)</code> is how Express identifies an error handler. It must be registered after all routes. All errors passed to <code>next(err)</code> converge here.
Custom error classes	Extend <code>Error</code> with <code>statusCode</code> and <code>isOperational</code> . Call <code>Object.setPrototypeOf(this, new.target.prototype)</code> to preserve <code>instanceof</code> checks in TypeScript. Domain subclasses (<code>NotFoundError</code> , etc.) make throw sites self-documenting.
Operational vs. programmer errors	Operational errors (expected, recoverable) surface their message to the client. Programmer errors (bugs) return a generic 500 in production; the stack trace goes to the log only.
<code>asyncHandler</code> wrapper	Catches rejected promises in async route handlers and calls <code>next(err)</code> automatically. Avoids repeating try/catch in every handler and is compatible with both Express 4 and Express 5.
Winston log levels	Use <code>error</code> for crashes, <code>warn</code> for client-facing operational errors, <code>info</code> for lifecycle events, <code>http</code> for access logs. Set <code>level</code> on the logger or per-transport to control verbosity.

Concept	Key Point
Winston formats	<code>json()</code> for production (machine-readable ingestion); <code>colorize() + printf()</code> for development (human-readable terminal). Switch via <code>NODE_ENV</code> .
Morgan integration	Stream Morgan output to Winston's <code>http</code> level via a <code>{ write }</code> object. This consolidates HTTP access logs and application logs into a single pipeline.
Global handlers	<code>process.on('uncaughtException')</code> and <code>process.on('unhandledRejection')</code> catch failures outside Express. Log and call <code>process.exit(1)</code> — do not attempt recovery. A process manager handles the restart.

Interview Questions

1. Express has three-argument middleware and four-argument middleware. What is the difference, and how does Express distinguish between them?

Expected answer: Three-argument middleware — `(req, res, next)` — is normal request-processing middleware. Four-argument middleware — `(err, req, res, next)` — is an error handler. Express inspects the function's `.length` property at registration time. If the length is exactly 4, Express treats it as an error handler and only calls it when an error has been passed to `next(err)`.

2. What does the `isOperational` flag on a custom error class represent, and why does it matter?

Expected answer: `isOperational: true` means the error is an expected, predictable failure — a user not found, an invalid field value. It is safe to send the error message to the client. `isOperational:`

`false` (or the absence of the flag, i.e., a plain `Error`) indicates a programmer error or an unanticipated runtime failure. In that case the client receives a generic 500 message to avoid leaking internal implementation details, while the full stack trace is written to the server log.

3. Why must `Object.setPrototypeOf(this, new.target.prototype)` be called in a TypeScript class that extends `Error`?

Expected answer: TypeScript compiles class inheritance to ES5 prototype chains when targeting older environments. The TypeScript compiler does not correctly restore the prototype chain when extending built-in classes like `Error`. Without the `Object.setPrototypeOf` call, `err instanceof NotFoundError` evaluates to `false` even though the object was constructed with `new NotFoundError()`, because its prototype chain points to `Error.prototype` rather than `NotFoundError.prototype`.

4. An async route handler in Express 4 throws an error inside a Promise. The error does not reach the centralised error middleware. What is the most likely cause and fix?

Expected answer: In Express 4, unhandled promise rejections inside route handlers are not automatically forwarded to error middleware. The fix is either to wrap the handler in a try/catch and call `next(err)` in the catch block, or to use an `asyncHandler` wrapper that calls `Promise.resolve(fn(...)).catch(next)`. Express 5 handles this automatically, but the explicit wrapper is compatible with both versions.

5. Your application runs in production with Winston's log level set to `'warn'`. A developer reports that Morgan access logs are not appearing. Why, and how do you fix it?

Expected answer: Morgan is configured to stream output to Winston at the `http` level, which has a numeric severity of 3. The logger's level of `'warn'` (severity 1) filters out anything below severity 1, which includes `http`. To restore access logs, set `LOG_LEVEL=http` (severity 3) in the production environment, or configure a dedicated Winston transport at level `http` with its own output file.

6. What is the difference between an `uncaughtException` and an `unhandledRejection` in Node.js?

Expected answer: An `uncaughtException` is a synchronous error that was thrown but not caught by any try/catch block in the call stack. An `unhandledRejection` is a promise that settled as rejected

without a corresponding `.catch()` handler or enclosing `try/catch` in an `async` function. Both escape Express's error middleware because they occur outside the request/response cycle managed by Express.

7. In a Kubernetes deployment, a team-mate configures Winston with both Console and File transports. The operations team complains they cannot see logs in their log aggregation tool. What is likely wrong?

Expected answer: Kubernetes collects logs from container stdout and stderr. If the Console transport is set to `silent: true` or suppressed, and logs are written only to files inside the container, the aggregator has nothing to collect — the files are ephemeral, local to the container, and invisible to the cluster's logging infrastructure. The fix is to remove the File transport and ensure the Console transport writes to stdout, which the aggregator captures.

8. Should an `uncaughtException` handler attempt to restore the application to normal operation and continue serving requests? Why or why not?

Expected answer: No. The Node.js documentation explicitly warns that an `uncaughtException` may leave the process in an undefined, inconsistent state — open file descriptors, corrupt in-memory data, broken database connections. Continuing to serve requests risks data corruption and unpredictable behaviour. The correct approach is to log the error, flush any buffers if possible, call `process.exit(1)`, and rely on a process manager (PM2, Kubernetes) to restart a clean instance.

Practical Assignment — Error Handling Pipeline

Objective: Build a TypeScript Express + TypeORM API with a complete, production-grade error-handling pipeline: custom error classes, a centralised four-argument error handler, Winston logging in both development and production formats, Morgan access logging, and global process-level handlers.

Estimated time: 45–60 minutes

1. **Scaffold the project.** Initialise a TypeScript project with Express and TypeORM connected to SQLite. Create a `Product` entity with `id` (auto-generated), `name` (string, not null), and `price` (decimal, not null).
2. **Create the error class hierarchy.** Implement `AppError` as the base class with `statusCode` and `isOperational`. Add at minimum: `NotFoundError` (404), `ValidationError` (400), and `ConflictError` (409). Verify that `new NotFoundError('Product')` instanceof `AppError` evaluates to `true`.
3. **Implement `asyncHandler`.** Write the wrapper and apply it to all async route handlers. Confirm that throwing an error inside a handler reaches the error middleware without a try/catch in the handler itself.
4. **Build the routes.** Implement `GET /products/:id` (throw `NotFoundError` if absent), `POST /products` (throw `ValidationError` if `name` or `price` is missing), and `DELETE /products/:id` (throw `NotFoundError` if absent).
5. **Configure Winston.** Create the logger with coloured `printf` format for development and `json()` for production. Switch format via `NODE_ENV`. Start the server and confirm that `logger.info('Server started')` appears in the expected format.
6. **Integrate Morgan.** Stream Morgan's output to Winston at the `http` level. Use the `'dev'` preset for development. Make three requests and confirm access log lines appear alongside application log lines.
7. **Implement the centralised error handler.** Handle `AppError` instances with their status code; return a generic 500 for all others. In development, include the stack trace in the 500 response. Register it last in `app.ts`.
8. **Add global process handlers.** Register `uncaughtException` and `unhandledRejection` in `server.ts` before starting the HTTP server. Both should log via Winston and call `process.exit(1)`.

Output:

```
POST /products {}           → 400 { "error": "name and price are required." }
GET /products/999          → 404 { "error": "Product not found." }
GET /products/1            → 200 { "id": 1, "name": "Widget", "price": "9.99" }
DELETE /products/1         → 204 No Content

// Winston console output (development):
10:05:01 [info] Server listening on port 3000
```

```
10:05:04 [http] POST /products 400 1.2 ms - 38
10:05:05 [warn] name and price are required. {"statusCode":400,"path":"/products","method":"POST"}
10:05:07 [http] GET /products/999 404 0.9 ms - 26
10:05:07 [warn] Product not found. {"statusCode":404,"path":"/products/999","method":"GET"}
```

Bonus Challenge: Add a `GET /products` route that accepts an optional `?minPrice=` query parameter. If the value is provided but is not a valid positive number, throw a `ValidationError`. Then deliberately introduce an `unhandledRejection` in a `setTimeout` callback (e.g., `setTimeout(() => Promise.reject(new Error('Simulated rejection')), 3000)`) at the bottom of `server.ts`. Start the server, wait three seconds, and confirm that the global handler logs the rejection and exits the process.

Note — Additions and Changes to Syllabus Coverage:

- **Added:** The `asyncHandler` wrapper utility. The syllabus lists "centralised error middleware" without addressing the practical problem of async route handlers not forwarding rejections to it in Express 4. This pattern is essential for making the centralised handler actually work and is universally used in production Express codebases.
- **Added:** The operational vs. programmer error distinction. The syllabus mentions "custom error classes" without elaborating on why the distinction matters or how it drives different response and logging behaviour. This is the key design decision behind the pattern.
- **Added:** The `Object.setPrototypeOf` caveat for TypeScript class inheritance from built-ins. This is a well-known TypeScript gotcha that silently breaks `instanceof` checks without an error message — important to flag explicitly.
- **Added:** Container/Kubernetes caveat for file transports. The syllabus does not mention deployment context for logging decisions, but students will encounter this immediately when deploying containerised applications.
- **TypeScript:** All code uses TypeScript as per the instruction added 1 March 2026.